Generative Representational Instruction Tuning

Niklas Muennighoff Hongjin Su Liang Wang Nan Yang
Furu Wei Tao Yu Amanpreet Singh Douwe Kiela
Contextual AI **The University of Hong Kong** **Microsoft Corporation**
arXiv:2402.09906v2

자연어처리팀 박산희,김준철,조해창,김세진,박주혜

Power of Embedding Models

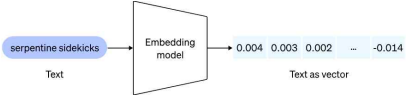
Text Embedding Use Cases

What are embeddings?

OpenAI's text embeddings measure the relatedness of text strings. Embeddings are commonly used for:

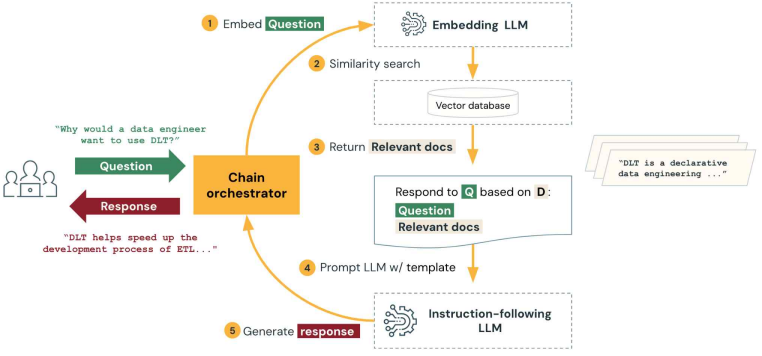
- **Search** (where results are ranked by relevance to a query string)
- **Clustering** (where text strings are grouped by similarity)
- **Recommendations** (where items with related text strings are recommended)
- **Anomaly detection** (where outliers with little relatedness are identified)
- **Diversity measurement** (where similarity distributions are analyzed)
- **Classification** (where text strings are classified by their most similar label)

An embedding is a vector (list) of floating point numbers. The distance between two vectors measures their relatedness. Small distances suggest high relatedness and large distances suggest low relatedness.



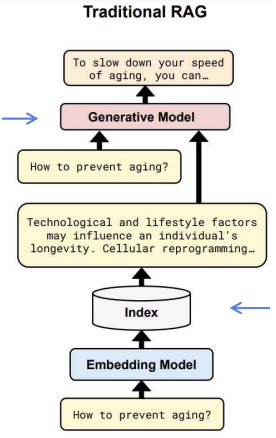
- Reducing embedding dimensions
- Question answering using embeddings-based search
- Text search using embeddings
- Code search using embeddings
- Recommendations using embeddings
- Data visualization in 2D
- Embedding as a text feature encoder for ML algorithms
- Classification using the embedding features
- Zero-shot classification
- Obtaining user and product embeddings for cold-start recommendation
- Clustering

Text Embeddings into the Generative Paradigm

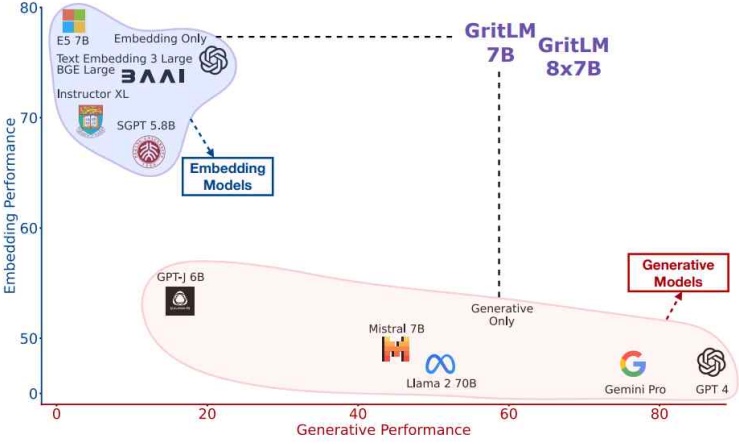


Seperate Embedding and Generative Model

Generative models lead to poor performance for text embedding



efficiency for high dimensitionality and need of high precision


80
70
60
50
0
Embedding Performance
E5 7B
Embedding Only
Text Embedding 3 Large
BGE Large
Instructor XL
SGPT 5.8B
Embedding Models
GritLM 7B
GritLM 8x7B
GPT-J 6B
Generative Only
Generative Models
Mistral 7B
Llama 2 70B
Gemini Pro
GPT 4
0
20
40
60
80
Generative Performance

Related Work

- Embedding Models
  - Infersent, SBERT → 단어 -> 문장 임베딩 퀄리티 개선
    - Embedding of words and sentences at good quality
  - SentT5, GTR → 쿼리/패시지 구조 길이 및 분포에 맞게 특화
    - For strong performance, models are specialized for symmetric and asymmetric tasks
  - E5, GTE, OpenAI Text-large etc.
    - Embedding models have been able to unify symmetric and asymmetric tasks into a single model → 하나의 모델로 통합

Related Work

- Generative Models
  - Model tailored to a single task (translation and question answering)

    → **특정 태스크에 맞게 생성 모델 디자인**
  - Multiple generative tasks are unified as form of question answering

    → **여러 태스크를 QA 형식으로 표현 통일하려는 시도**
  - Finetuning LLMs on instructions : any generation task with strong results

    → **Instruction tuning with LLMs의 발전으로 여러 생성 태스크의 강력한 성능**

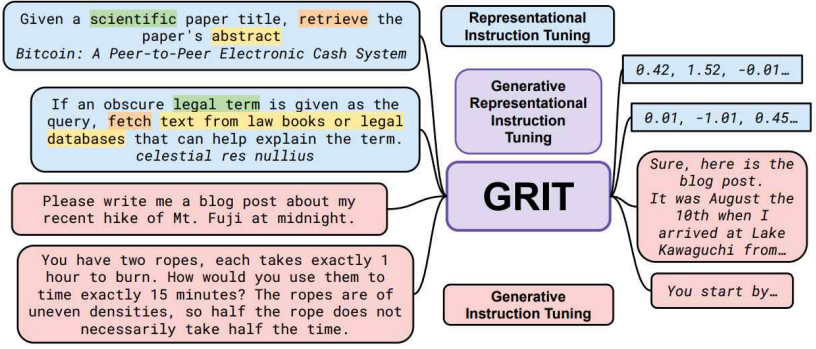
Related Work

- Stream
  - Single model that handles any task within its stream

→ **본 연구에서 두 태스크인 embedding과 generative task를 한번에 다루도록 하는 계기**

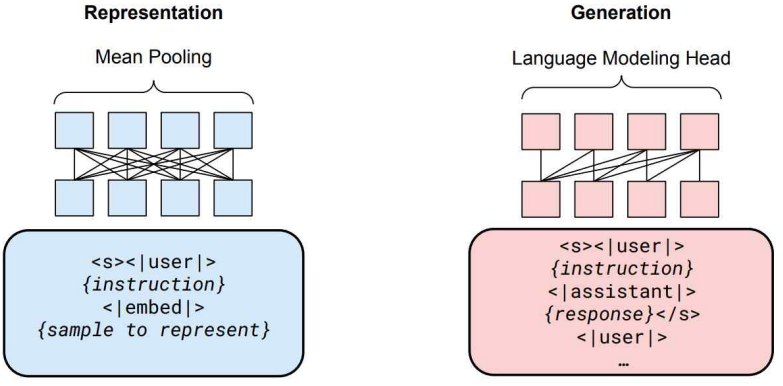
  - LLMs have shown promise for text embeddings (Niklas 2022; Neelakantan 2022;Ting2023;Li2023)

→ **LLMs 기반 text embeddings 모델들의 등장**



→ **Embedding 및 Generative Tasks를 동시에 학습하도록 하는 "GRIT" 방법 소개**

GRIT



→ **GritLM attention 방법 : representation learning 시에는 bi-directional attention 사용**
generation tuning 시에는 causal attention 사용

GRIT

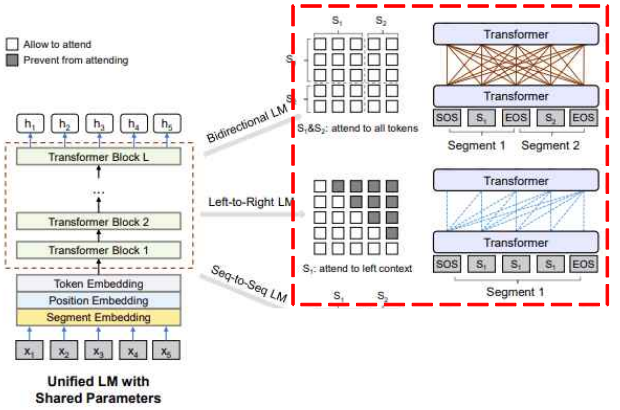
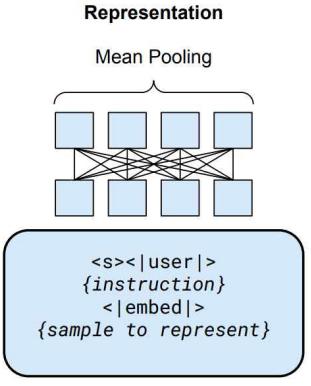


Figure 1: Overview of unified LM pre-training. The model parameters are shared across the LM objectives (i.e., bidirectional LM, unidirectional LM, and sequence-to-sequence LM). We use different self-attention masks to control the access to context for each word token. The right-to-left LM is similar to the left-to-right one, which is omitted in the figure for brevity.

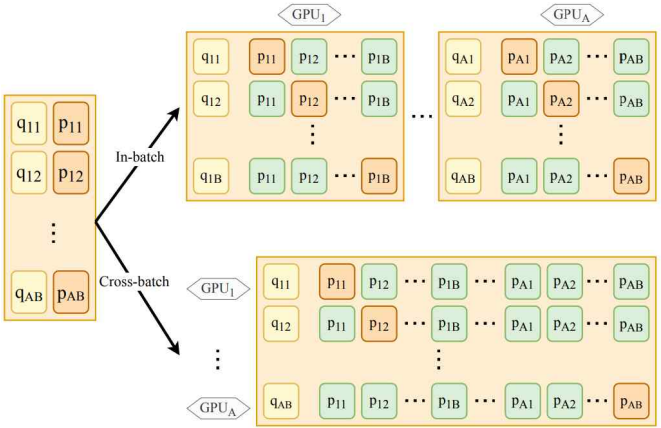
이미지 출처 : Deep Reinforced Self-Attention Masks for Abstractive Summarization

GRIT



→ **Representation 입력 : {"query": ["__instruction__", "__positive_doc__" "__negative_doc__"]}**

GRIT

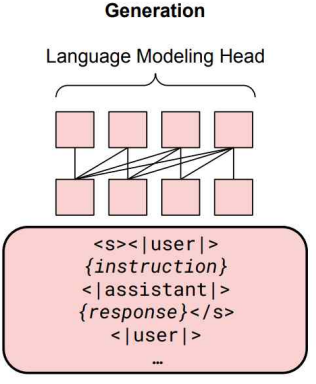


→ **Cross entropy loss with in-batch negatives**

$$\mathcal{L}_{\text{Rep}} = -\frac{1}{M}\sum_{i=1}^{M}\log\frac{\exp(\tau\cdot\sigma(f_\theta(q^{(i)}),f_\theta(d^{(i)})))}{\sum_{j=1}^{M}\exp(\tau\cdot\sigma(f_\theta(q^{(i)}),f_\theta(d^{(j)})))}$$

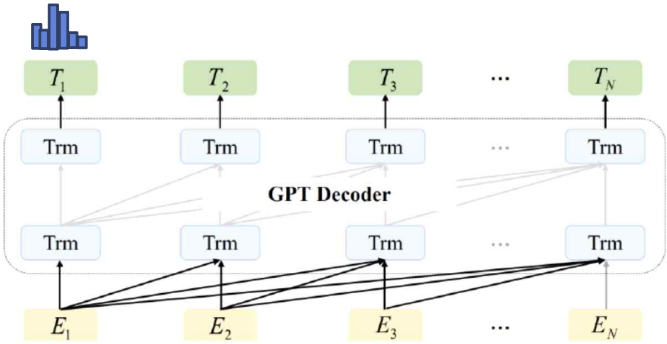
이미지 출처 : https://blog.reachsumit.com/posts/2023/03/pairing-for-representation/

GRIT



Generation 입력 : {"text": ["__instruction__""__question__", "__answer__" ...]}

GRIT



→ language modeling loss $\mathcal{L}_{\text{Gen}} = -\frac{1}{N}\sum_{i=1}^{N} \log P(f_{\theta,\eta}(x^{(i)})|f_{\theta,\eta}(x^{(<i)}))$

→ weighted sum loss $\mathcal{L}_{\text{GRIT}} = \lambda_{\text{Rep}}\mathcal{L}_{\text{Rep}} + \lambda_{\text{Gen}}\mathcal{L}_{\text{Gen}}$

이미지 출처 : https://hugrypiggykim.com/2019/08/29/researches-after-bert/

Q & A

Experiments

- Setup
  - Model : Mistral 7B and Mixtral 8x7B
  - Dataset : E5 dataset and Tulu 2 data
  - Training Details:
    - weighted sum two loss
    - Batch size : GritLM 7B -> embedding model : 2048 generative model : 256
    - 1235 steps fine-tuning, 2e-05 learning rate , warm-up step (0.03)
    - length limit -> embedding query : 256 embedding document: 2048; generation: 2048
  - Evaluation
    - MTEB -56 datasets (embedding) , Tulu, HumanEvalSynthesize (generation)

Main Results

Task (→) Metric (→) Dataset # (→)	CLF Acc. 12	Clust. V-Meas. 11	PairCLF AP 3	Rerank MAP 4	Retrieval nDCG 15	STS Spear. 10	Summ. Spear. 1	Avg. 56
Proprietary models♥								
OpenAI v3	75.5	49.0	85.7	59.2	55.4	81.7	29.9	64.6
Other Open Models♥								
Llama 2 70B	60.4	29.0	47.1	38.5	9.0	49.1	26.1	35.6
Mistral 7B	63.5	34.6	53.5	43.2	13.2	57.4	19.7	40.5
Mistral 7B Instruct	67.1	34.6	59.6	44.8	16.3	63.4	25.9	43.7
GPT-J 6B	66.2	39.0	60.6	48.9	19.8	60.9	26.3	45.2
SGPT BE 5.8B	68.1	40.3	82.0	56.6	50.3	78.1	31.5	58.9
Instructor XL 1.5B	73.1	44.7	86.6	57.3	49.3	83.1	**32.3**	61.8
BGE Large 0.34B	76.0	46.1	87.1	60.0	54.3	83.1	31.6	64.2
E5 Mistral 7B	78.5	50.3	**88.3**	60.2	56.9	**84.6**	31.4	66.6
GRITLM								
Gen.-only 7B	65.4	32.7	54.2	43.0	13.7	60.2	21.1	41.2
Emb.-only 7B	78.8	**51.1**	87.1	**60.7**	**57.5**	83.8	30.2	**66.8**
GRITLM 7B	**79.5**	50.6	87.2	60.5	57.4	83.4	30.4	**66.8**
GRITLM 8X7B	78.5	50.1	85.0	59.8	55.1	83.3	29.8	65.7

Not fine-tuned

Main Results

Table 2: **Generative performance of GRITLM and others**. We indicate parameter counts where available (B=billions). See Appendix D for dataset, setup, and metric details. ♥Results from Ivison et al. [64] except for numbers marked with ♦ which are from Touvron et al. [154] and † which are from us. For models that cannot be easily used as chat models, we set Alpaca to 0.

Dataset (→) Setup (→) Metric (→)	MMLU 0 FS EM	GSM8K 8 FS, CoT EM	BBH 3 FS, CoT EM	TyDi QA 1 FS, GP F1	HumanEval 0 FS pass@1	Alpaca 0 FS, 1.0 % Win	Avg.
Proprietary models♥							
GPT-4-0613	81.4	95.0	89.1	65.2	86.6†	91.2	84.8
Other Open Models♥							
GPT-J 6B	27.7	2.5	30.2	9.4	9.8	0.0	13.3
SGPT BE 5.8B	24.4	1.0	0.0	22.8	0.0	0.0	8.0
Zephyr 7B β	58.6	28.0	44.9	23.7	28.5	85.8	44.9
Llama 2 7B	41.8	12.0	39.3	51.2	12.8♦	0.0	26.2
Llama 2 13B	52.0	25.0	48.9	56.5	18.3♦	0.0	33.5
Llama 2 70B	64.5	55.5	66.0	**62.6**	29.9♦	0.0	46.4
Llama 2 Chat 13B	53.2	9.0	40.3	32.1	19.6†	91.4	40.9
Llama 2 Chat 70B	60.9	59.0	49.0	44.4	34.3†	94.5	57.0
Tülu 2 7B	50.4	34.0	48.5	46.4	24.5†	73.9	46.3
Tülu 2 13B	55.4	46.0	49.5	53.2	31.4	78.9	52.4
Tülu 2 70B	67.3	**73.0**	68.4	53.6	41.6	86.6	65.1
Mistral 7B	60.1	44.5	55.6	55.8	30.5	0.0	41.1
Mistral 7B Instruct	53.0	36.0	38.5	27.8	34.0	75.3	44.1
Mixtral 8x7B Instruct	**68.4**	65.0	55.9	24.3	**53.5**	**94.8**	60.3
GRITLM							
Emb.-only 7B	23.5	1.0	0.0	21.0	0.0	0.0	7.6
Gen.-only 7B	57.5	52.0	55.4	56.6	34.5	75.4	55.2
GRITLM 7B	57.6	57.5	54.8	55.4	32.8	74.8	55.5
GRITLM 8x7B	66.7	61.5	**70.2**	58.2	53.4	84.0	**65.7**

Ablations

Attention Emb Instruction	Attention Emb Sample	Attention Gen Instruction	Attention Gen Sample	Pooling	Emb	Gen
		Embedding Only				
Causal				Wmean	60.0	-
Causal	Bidirectional			Mean	61.0	-
Bidirectional				Mean	61.8	-
		Generative Only				
		Causal			-	**55.2**
		Bidirectional	Causal		-	50.7
		Unified				
Causal		Causal		Last token	61.2	53.0
Causal		Causal		Wmean	62.8	52.8
Bidirectional		**Causal**		**Mean**	**64.0**	52.9

(a) Attention and pooling ablations. Wmean is position-weighted mean pooling [104].

→ **Representation 학습 시, bi-directional attention이 효과적,**
generation 학습 시, causal attention 일관적으로 사용이 효과 ▲

Ablations

Variant	Emb	Gen
Mistral 7B	**54.6**	**22.4**
Llama 2 7B	48.2	20.8
GPT-J 6B	51.9	14.0

(b) Base model

Dataset	Emb
MEDI	64.0
MEDI2	64.7
E5	**66.0**

(c) Embedding dataset

Dataset	Gen
Tülu 2	**55.2**
OASST	37.7
UltraChat	47.4

(d) Generative dataset

Variant	Emb	Gen
No head	**62.7**	**49.2**
-> 1024	62.1	48.0

(e) Embedding head

BS Emb:Gen	Emb	Gen
256:256	63.2	**53.4**
4096:256	**64.2**	53.3

(f) Batch size (BS)

Precision	Emb	Gen
FP32	66.3	52.4
BF16*	**66.5**	**55.0**

(g) Precision

IBN origin	Emb	Gen
Any dataset	**66.0**	50.9
Same dataset	**66.0**	**51.1**

(h) In-batch negatives (IBN)

Format	Gen
Tülu 2	**55.2**
Zephyr β	49.0

(i) Format

Tokens	Emb	Gen
512	64.1	52.2
2048	**64.7**	**53.8**

(j) Emb training max tokens

Gen loss type	$\mathcal{L}_{\text{Rep}}/\mathcal{L}_{\text{Gen}}$	Emb	Gen
Token	2.4	66.1	54.4
Token	6.0	66.5	55.0
Mix (32 -> 8)	4.1	**66.7**	**55.4**

Gen loss type	AlpacaEval
Mix (4 -> 64)	67.6
Mix (32 -> 8)	**74.7**

→ **베이스모델, 학습 데이터세트, 각 배치 사이즈,**
태스크 간 손실 비율 등 실험을 통해 생성 및 임베딩 태스크에 최적 선택

RAG with GRIT

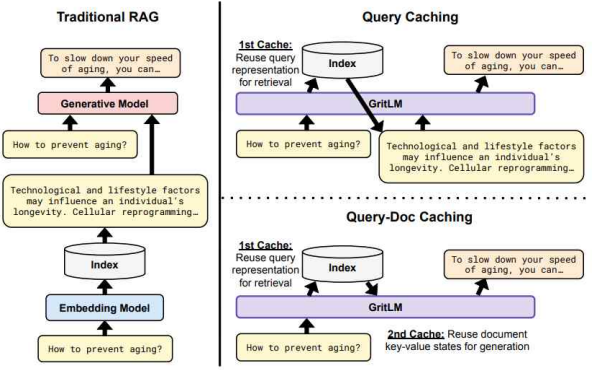
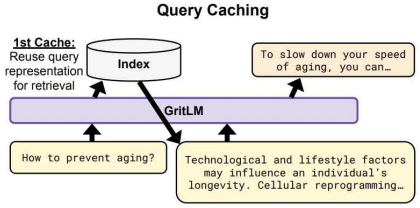
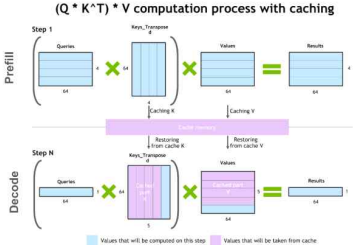


Figure 4: **RAG with GRIT.** *Left:* Traditional Retrieval-Augmented Generation (RAG) relies on a separate embedding model and generative model. *Right:* GRITLM simplifies RAG as it handles both embedding and generation. Query Caching removes the duplicate forward pass of the query by reusing its representation. Query-Doc Caching also removes the forward pass on the document during inference, as the cached index also stores the document key-value states.

→ **RAG 구조에서 GRIT을 활용, query/doc caching 통해 추론속도 개선**

RAG with GRIT





Query caching

- **embedding forward pass 시 query의 key-value 상태 저장, generative pass 시 사용**

Query Doc Caching

- **query caching과 doc caching 결합,**
- **doc cahcing은 embedding forward 시 index에 document key-value 상태 함께 저장,**
- **generative pass 시 사용**

RAG with GRIT

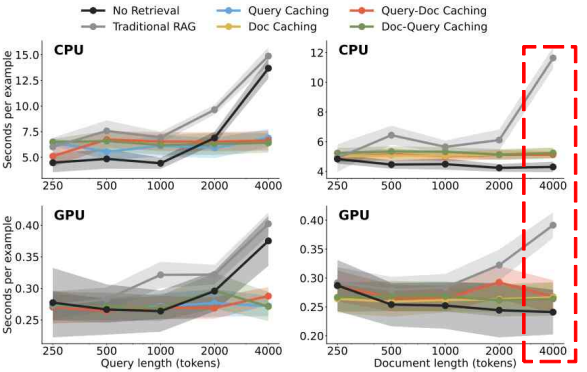


Figure 5: **Inference latency of RAG with GRITLM 7B.** When benchmarking scaling query length (left), document length is fixed at 1, whereas query length is fixed at 1 when scaling document length (right). In addition to the query/doc lengths, the formatting and prompt take up around 40 tokens. We visualize the standard deviation across 100 runs as the shaded area. For each approach, we generate 16 tokens.

→ **when 4000 tokens are generated, Query Caching is 54% and 33% faster on CPUs and GPUs. For Doc Caching, it is 63% and 31%.**

Table 7: **RAG benchmarking on Natural Questions with GRITLM 7B.** For RAG, the retrieved context is simply placed in the context of the language model in contrast to our caching alternatives (Figure 4). CPU and GPU latencies are measured on an "Intel(R) Xeon(R) Platinum 8481C CPU @ 2.70GHz" and one "NVIDIA H100 80GB HBM3", respectively. Sample A has a query of 1 token and a document of 4000 tokens, and sample B is the inverse. For each approach, we generate 16 tokens. Storage consists of the index and passages, except for Doc Caching variants where it is the index and key-value states. The index is stored in float32, while key-value states are stored in bfloat16.

	Match (0-shot, ↑)	CPU Latency (s, ↓) Sample A	CPU Latency (s, ↓) Sample B	GPU Latency (s, ↓) Sample A	GPU Latency (s, ↓) Sample B	Storage (↓)
No RAG	21.00	4.3 ± 0.36	13.69 ± 1.0	0.24 ± 0.04	0.38 ± 0.04	**0GB**
Query then document prompt						
RAG	30.50	11.64 ± 0.74	14.88 ± 0.87	0.39 ± 0.02	0.40 ± 0.02	43GB
Query Caching	25.46	18.30 ± 0.76	6.87 ± 0.89	0.44 ± 0.03	**0.27 ± 0.02**	43GB
Query-Doc Caching	21.63	**5.12 ± 0.23**	6.62 ± 0.97	0.27 ± 0.03	0.29 ± 0.01	30TB
Document then query prompt						
RAG	30.47	14.18 ± 1.01	15.33 ± 0.87	0.39 ± 0.01	0.4 ± 0.01	43GB
Doc Caching	**33.38**	5.25 ± 0.34	23.23 ± 1.05	0.27 ± 0.03	0.45 ± 0.02	30TB
Doc-Query Caching	18.39	5.23 ± 0.37	**6.41 ± 0.96**	**0.26 ± 0.03**	**0.27 ± 0.02**	30TB

The key-value states take up a lot of storage, as for each batch they consist of two tensors of shape (batch size, number of heads, sequence length, dimension per head)

Conclusion

- GRIT unify text embedding and generation into a **single model**
  - GritLM, GritLM 7B achieves state-of-the-art performance on MTEB
  - GritLM 8x7B boasts strong embedding performance thanks tothe Grit approach

- GritLM allows caching opereations learning to **inference speed-ups** of > 60%
  - GRIT unifiy the retriever and reader into a single model
  - no performance loss with doc caching for long sequences

Q & A

Thank You